

CI de Nota Automática

Como fazer — e como foi feito

Guia para aplicar a mesma avaliação automatizada nos seus projetos

GitHub Actions · React Native / Expo · SQLite

Este documento descreve uma esteira de avaliação automatizada construída para a disciplina de PAM I e já aplicada a 12 projetos de grupos reais. Ele tem duas partes: como a esteira funciona e foi implantada, e o passo a passo para outro professor replicá-la em suas próprias turmas.

A ideia em uma frase

Um script lê o código do projeto dos alunos, confere 40 critérios das três fases da disciplina, calcula uma nota de 0 a 55 pontos e publica um badge e um checklist direto no README do repositório — automaticamente, a cada push. Ninguém precisa abrir o projeto, ler 40 arquivos ou anotar nada: o número aparece no topo do README.

O que você vai encontrar aqui

Parte	Conteúdo
Parte 1	O que a esteira faz e como ela foi desenhada (conceitos, arquitetura, os 4 arquivos)
Parte 2	Como foi feito na prática, com as armadilhas reais que quase bloquearam a implantação
Parte 3	Receita: como aplicar no seu projeto, passo a passo, do zero ao badge funcionando
Parte 4	Referência rápida: os 40 critérios, as faixas de nota, comandos e troubleshooting
Parte 5	Limitações e cuidados: o que a nota mede e o que ela não mede
Apêndice A	Os 12 projetos da turma e as notas reais obtidas

Os números desta implantação

Estes são os números reais da primeira rodada completa, medida nos 12 repositórios da turma. Servem de referência para calibrar a sua: se a sua distribuição vier muito diferente, vale investigar o config antes de suspeitar dos alunos.

Indicador	Valor
Projetos avaliados	12 grupos
Critérios verificados	40 (10 na Fase 1, 15 na Fase 2, 15 na Fase 3)
Pontuação máxima	55 pontos (10 + 15 + 30)
Custo	zero — o GitHub Actions é gratuito para repositórios públicos
Dependências do script	zero — Node.js puro, nada para instalar
Config por projeto	12 arquivos, escritos à mão, um por arquitetura
Projetos que já mergearam a esteira	6 de 12 no momento da escrita deste guia

Como usar este documento

-
- **Quer aplicar?** Vá direto para a Parte 3 (receita). A Parte 4 serve de consulta durante a execução.
Quer entender o porquê? Leia as Partes 1 e 2. A 2.3 é a mais importante: são as armadilhas que custaram mais tempo na implantação real.
- **Quer ajustar a avaliação?** A Parte 4.1 traz os 40 critérios um a um, com o peso de cada um — é onde se mexe quando a disciplina evolui.
- **Quer saber se pode confiar no número?** A Parte 5 é obrigatória. Ela lista o que a nota não mede, incluindo o que um aluno conseguiria passar sem ter feito o trabalho.

A decisão de ethos, antes de tudo

Esta esteira foi desenhada para NÃO alterar o código do aluno. Ela lê o projeto e escreve um relatório; nunca sobrescreve o que o grupo fez. Respeitar a arquitetura do aluno é requisito de projeto, não cortesia: um avaliador que reescreve o trabalho do grupo deixa de ser justo. Por isso a adaptação a cada projeto é feita no config, e nunca no código do aluno.

Sumário

Parte 1 — Como a esteira funciona	4
1.1 O problema que ela resolve	4
1.2 A arquitetura em quatro arquivos	4
1.3 Como cada critério é testado	6
1.4 A pontuação e as faixas de conceito	6
1.5 O que o aluno vê	8
Parte 2 — Como foi feito	9
2.1 O que já existia e o que precisei construir	9
2.2 A ordem das decisões	9
2.3 Cinco armadilhas que quase pararam tudo	10
Parte 3 — Como aplicar no seu projeto	12
3.1 Pré-requisitos	12
3.2 Passo a passo, do zero ao badge	12
3.3 Adaptando às arquiteturas mais comuns	13
3.4 Testes antes de abrir o PR	13
Parte 4 — Referência rápida	14
4.1 Os 40 critérios, item a item	14
4.2 Comandos do dia a dia	15
4.3 Troubleshooting	15
Parte 5 — Limitações e cuidados	16
5.1 O que a nota mede	16
5.2 O que ela não mede	16
5.3 Falsos positivos e falsos negativos	16
5.4 Checklist de revisão humana	16
Apêndice A — Os 12 projetos e suas notas	17

Parte 1 — Como a esteira funciona

Esta parte descreve a arquitetura. Se você só quer aplicar, pule para a Parte 3 e volte aqui quando algo der errado.

1.1 O problema que ela resolve

A disciplina tem três fases e um produto final em grupo. A avaliação natural de um trabalho em grupo acaba sendo: abrir o repositório, procurar a tela de listagem, procurar o storage, procurar o banco, contar o que existe e o que não existe. Em 12 projetos, são 40 perguntas repetidas 12 vezes.

O material automático disponível no GitHub é ótimo para testar se o código compila — e inútil para a pergunta que importa: "esse grupo entregou a Fase 3?". Testes automatizados verificam se o software funciona. Não verificam se o escopo pedido foi cumprido.

A saída foi um analisador estático com um checklist de 40 itens derivado das fichas das fases, executado pelo GitHub a cada push, com o resultado publicado dentro do próprio repositório do grupo. Três decisões moldaram o resto:

- **Sem alteração do código do aluno.** A esteira lê e escreve só os próprios relatórios. Isso é requisito: um avaliador que reescreve o trabalho do grupo deixa de ser justo.
- **Resultado no lugar onde o grupo já olha.** O grupo abre o repositório; o badge está no topo do README.
- Nada de pedir que o aluno baixe um .zip na aba Actions.
Nota auditável. Todo item ganha uma "prova" — o arquivo onde a evidência foi encontrada. O professor julga sobre a evidência, não sobre um número.

1.2 A arquitetura em quatro arquivos

A esteira inteira cabe em quatro arquivos dentro de .github/. Não há dependência a instalar, não há banco, não há servidor.

[.github/pam-ci.config.json — o único arquivo que você ajusta por projeto](#)

Diz ao analisador onde está cada peça do projeto. É o arquivo que precisa ser escrito à mão para cada turma, porque cada grupo organiza os arquivos do seu jeito:

```
{
  "grupo": "TechControl (equipamentos)",
  "raiz": ".",
  "principal": "App.js",
  "lista": "src/screens/EquipmentListScreen.js",
  "form": "src/screens/EquipmentFormScreen.js",
  "detalhe": "src/screens/EquipmentDetailScreen.js",
  "dados": [
    "src/data/equipments.js"
  ],
  "storage": "src/data/equipments.js",
  "sqlite": ""
}
```

O campo raiz existe porque alguns grupos têm o app dentro de uma subpasta (educamirim/app, app/), e outros na raiz. Nos 12 projetos desta turma, cinco usavam subpasta.

Por que o config importa tanto

Os critérios da Fase 1 (existe tela de listagem, existe formulário, existe detalhe) conferem a existência de um caminho de arquivo. Sem o config, um projeto com as telas em `src/screens/` seria pontuado como se não tivesse telas. É o arquivo que adapta a esteira à arquitetura de cada grupo — e a razão de ela não "reescrever" o projeto.

`.github/scripts/pam-ci.mjs` — o analisador

Node.js puro, sem dependências. Lê o `package.json`, varre todos os arquivos `.js`, `.jsx`, `.ts` e `.tsx` do projeto (ignorando `node_modules`, `.git`, `.expo`, `android`, `ios` e `build`), e para cada um dos 40 critérios testa se o padrão esperado existe. Ao final escreve dois arquivos: `nota.md` (relatório para humanos) e `nota.json` (dados estruturados, útil para automação e gráficos).

`.github/scripts/publicar-nota.mjs` — o publicador

Lê o `nota.json` e escreve o resultado dentro do repositório: injeta o bloco com os badges no `README.md` e cria o `NOTA.md` na raiz. É separado do analisador por um motivo prático: o analisador roda em qualquer contexto (inclusive na sua máquina, sem GitHub), e só o publicador depende do GitHub.

`.github/workflows/pam-ci.yml` — o orquestrador

O workflow do GitHub Actions. Dispara em push na main, em pull_request e manualmente. Faz checkout, instala o Node, roda o analisador, publica o resumo na aba Actions, sobe um artefato e — só em push na main — chama o publicador e commita o resultado. O detalhe do "somente em push na main" está explicado na Parte 2.

Fluxo completo de um push na branch main:

```
push na main
-> workflow dispara
  -> pam-ci.mjs      le o codigo, confere 40 criterios, gera nota.md + nota.json
  -> resumo          nota.md vai para a aba Actions (visivel sem baixar nada)
  -> artefato        nota-pam.zip com nota.md + nota.json (download, opcional)
  -> publicar-nota   escreve o bloco do badge no README.md e cria o NOTA.md
  -> commit         "docs: atualiza a nota automatica [skip ci]"
-> badge do README ja mostra a nota nova
```

1.3 Como cada critério é testado

O analisador usa três estratégias, do mais confiável ao mais frágil. Conhecer essa hierarquia é o que permite corrigir um critério mal calibrado sem reescrever o sistema.

Estratégia 1 — o arquivo existe

```
ok: fileExists(CONFIG.lista)
// e a prova exibida no relatório:
prova: CONFIG.lista // -> src/screens/EquipmentListScreen.js
```

É o critério mais confiável: um arquivo existe ou não existe. Serve para os itens da Fase 1 que tratam de estrutura (existe tela de listagem, existe formulário, existe detalhe). Depende inteiramente do config estar correto.

Estratégia 2 — o padrão existe em algum arquivo do projeto

```
ok: grepFiles(/CREATE TABLE\s+IF NOT EXISTS/i).length > 0
prova: grepFiles(/CREATE TABLE\s+IF NOT EXISTS/i)[0] // -> src/database.js
```

É a estratégia mais usada (a maioria dos 40 itens). `grepFiles` percorre todos os arquivos de código e devolve os que casam com a expressão regular. O primeiro resultado vira a "prova" exibida no relatório, o que torna cada ponto verificável por mão.

Estratégia 3 — as dependências resolvem (a mais elaborada)

```
const re = /(?:(?:import|export)\s+(?:['"]*\s+from\s+)?|require\s*\(\s*['"]*(['"]*)+['"]*)/g;
// extrai todo import/require do projeto e compara com package.json
ok: unresolvedImports().length === 0
```

Este é o critério mais sofisticado da Fase 1 e o que mais frequentemente pega bug real. Ele extrai todo `import` e todo `require` do projeto e confere se cada pacote está declarado no `package.json`. Serve para pegar o clássico "usou `AsyncStorage` mas esqueceu de instalar o pacote": o aluno ganha o ponto da tela, mas o app quebra ao abrir. É o único critério que detecta erro de configuração real em vez de ausência de código.

A Fase 3 exige a cadeia completa de CRUD

Um aluno que copiasse um `SELECT` solto e declarasse a fase concluída não deve passar. Por isso a Fase 3 não verifica "tem SQL", verifica a cadeia inteira:

- >
- > cria a tabela (`CREATE TABLE IF NOT EXISTS`),
- > insere (`INSERT INTO`), consulta (`SELECT`), atualiza (`UPDATE`), exclui (`DELETE FROM`),
- > filtra (`WHERE`) e abre o banco (`openDatabaseAsync`),
e então três itens de integração: a lista carrega do banco, o formulário salva com `runAsync`, o detalhe busca com `WHERE`.

Cada item da Fase 3 vale 2 pontos, porque a fase é a mais pesada da disciplina.

1.4 A pontuação e as faixas de conceito

Fase	Cr�terios	Peso por item	Total
Fase 1 — Estrutura do projeto	10	1 ponto	10 pontos
Fase 2 — Persist�ncia com AsyncStorage	15	1 ponto	15 pontos
Fase 3 — Banco de dados SQLite	15	2 pontos	30 pontos
Total	40	—	55 pontos

A Fase 3 tem o dobro do peso porque   o conte do novo e o que mais distingue um projeto completo de um parcial. A convers o para conceito usa o percentual de pontos:

Conceito	Faixa	Significado
I — Insuficiente	0% a 24%	N�o entregou o essencial da disciplina
R — Regular	25% a 49%	Entregou a base, falta conte�do
B — Bom	50% a 74%	Entrega as tr�s fases com alguma profundidade
MB — Muito bom	75% a 100%	Projeto completo, todas as fases bem desenvolvidas

O badge usa a mesma faixa, com uma cor: I vermelho, R laranja, B azul, MB verde. A cor   o que o grupo v  primeiro.

1.5 O que o aluno vê

Três lugares, em ordem de visibilidade:

Onde	O que aparece	Quando
README.md do repositório	Bloco "Nota atual": 2 badges (status do CI e nota com conceito e cor), mais uma tabela com os pontos de cada fase	A cada push na main, escrito pelo CI
NOTA.md na raiz	Checklist completo dos 40 itens, com [x] e [] e a "prova" de cada um	A cada push na main, escrito pelo CI
Aba Actions	O relatório da rodada no campo Summary, sem precisar baixar nada	A cada execução
Artefato nota-pam	nota.md e nota.json em .zip, para quem quiser processar dados	A cada execução, sobrescrevendo o anterior

O bloco do README fica entre dois marcadores de comentário:

```
<!-- PAM-CI-NOTA-INICIO -->
### Nota atual (automática) – TechControl (equipamentos)
[![CI](...badge.svg)] [![Nota](https://img.shields.io/badge/Nota%20PAM%20I-B-yellow)]
**B** – Bom · **56%** (31/55 pontos) · atualizado em 2026-10-05 23:29
| Fase | Pontos |
| Fase 1 – Estrutura | 10/10 |
<!-- PAM-CI-NOTA-FIM -->
```

O publicador é idempotente e cirúrgico

O publicador reescreve apenas o conteúdo entre os dois marcadores. Se eles ainda não existirem, insere o bloco logo abaixo do primeiro título (#) do README. Rodar duas vezes não duplica nada e nunca toca no resto do README do grupo. Foi testado explicitamente para isso antes de subir nos 12 repositórios.

Parte 2 — Como foi feito

Esta parte é o registro honesto do processo, incluindo o que deu errado. Se você vai fazer o mesmo, a lista de armadilhas da seção 2.3 economiza mais tempo do que qualquer outra parte deste documento.

2.1 O que já existia e o que precisei construir

O ponto de partida: 12 repositórios de grupos, cada um com uma arquitetura diferente, e uma disciplina já montada com 20 aulas e três fases avaliadas. Existia um gerador de nota (o artefato antigo na aba Actions), mas ele falhava em três requisitos:

Requisito	Situação original	Decisão tomada
O grupo precisa ver a nota sem esforço	Nota escondida: dentro de um .zip no fim da página da rodada, com 0 downloads	Publicar no README com badge + NOTA.md
Não posso alterar o trabalho do aluno	Risco real ao distribuir um gerador	Adaptar por config; escrever só relatórios
Precisa funcionar em 12 arquiteturas distintas	Cada grupo estruturou os arquivos do seu jeito	Um config por projeto, 12 configs escritos à mão

A segunda linha foi a mais importante na prática. Um gerador "universal" teria que reescrever o projeto para normalizar a arquitetura — exatamente o que não se deve fazer com trabalho de aluno. Adaptar um config por projeto é mais lento, mas preserva o trabalho de cada grupo intacto.

2.2 A ordem das decisões

- 1 Extrair os critérios das fichas das fases**
Os 40 itens não foram inventados: saíram das fichas da Fase 1, 2 e 3 da disciplina, convertendo cada requisito em uma pergunta verificável no código.
- 2 Escrever o analisador como script de Node puro**
Sem dependências, para rodar tanto no GitHub Actions quanto na máquina do professor, para depurar e para testar antes de abrir qualquer PR.
- 3 Calibrar em um projeto real, não em exemplo fictício**
O primeiro projeto usado para calibração foi o equipcontrol, do grupo TechControl. Calibrar em código real expõe imediatamente os casos que um exemplo inventado esconderia (telas em subpasta, dependência só depois de a nota estar confiável veio o publicador, porque publicar um número errado de forma visível é pior tentando, seed fora do lugar).
- 4 Adicionar a publicação no README**
Escrever um config por grupo e testar cada um localmente
- 5 Escrever um config por grupo e testar cada um localmente**
Antes de abrir qualquer PR, o script foi rodado na máquina contra o main de cada um dos 12 projetos, confirmando que a nota fazia sentido e que o README não era destruído.
- 6 Abrir PR, um por projeto, e esperar os grupos mergearem**
O CI é instalado no repositório de cada grupo via pull request. Nenhum push direto foi feito no repositório de nenhum aluno.

2.3 Cinco armadilhas que quase pararam tudo

Estas são as lições que custaram mais tempo. Elas estão aqui porque qualquer pessoa replicando a esteira vai encontrá-las.

Armadilha 1 — o token sem escopo de workflow (a que mais custou tempo)

Para enviar uma branch que contenha arquivos `.github/workflows/` para um repositório usando a linha de comando do GitHub, o token precisa do escopo `workflow`. Sem ele, o GitHub rejeita o push com uma mensagem que parece de rede:

```
! [remote rejected] ci/nota-visivel -> ci/nota-visivel
(refusing to allow an OAuth App to create or update workflow
`.github/workflows/pam-ci.yml` without `workflow` scope)
```

O pior detalhe: se você encadeia o comando do git com um pipe (por exemplo `git push ... | tail`), o código de saída que você testa é o do último comando do pipe, não o do git. O script reporta sucesso, o push não aconteceu, e o erro some. Foi exatamente o que aconteceu nos primeiros 8 PRs: o script dizia "branch enviada" e nenhum branch existia no fork.

A lição

Nunca confie em `git push` dentro de um pipe sem checar o código de saída do git em si (use `PIPESTATUS`, ou redirecione a saída para um arquivo e leia depois). E para enviar workflows por SSH, use uma chave — a chave SSH não tem essa restrição de escopo que o token OAuth tem.

Armadilha 2 — o loop infinito de commits

O workflow escreve no README e faz commit. Se esse commit disparar o workflow de novo, ele reescreve o README e commita de novo, para sempre. Duas proteções resolvem, e ambas são necessárias:

- **[skip ci] na mensagem do commit** — o GitHub Actions não dispara workflow para commits com esse marcador.
- **checar quem é o autor** — a etapa de publicação só roda se o autor do push não for o próprio bot (`github.actor != github-actions[bot]`).

Sem a segunda proteção, um push do próprio bot reativaria o fluxo. Com as duas, o fluxo é idempotente.

Armadilha 3 — pull request de fork não consegue escrever

A etapa que escreve no README só roda em push na branch `main` — nunca em `pull_request`. Isso não é limitação do script, é do GitHub: em PRs que vêm de fork, o token é somente-leitura, e commitar num PR do próprio repositório empurraria commits na branch de trabalho do aluno. A solução foi preparar o PR: o branch já sobe com o README e o `NOTA.md` gerados, para o aluno ver a prévia do que vai acontecer. O `main` recebe a versão autoritativa no momento do merge.

Armadilha 4 — a nota some se ninguém abre o repositório

A primeira versão da esteira só produzia artefato (o .zip na aba Actions). Auditando os 12 projetos, o resultado foi constrangidor: os artefatos existiam, mas tinham 0 downloads. A nota estava tecnicamente disponível e praticamente invisível. Daí a decisão de publicar no README — não como complemento do artefato, mas como o canal principal.

Armadilha 5 — config errado dá nota errada com muita confiança

O config diz onde está cada tela. Se aponta para um caminho errado em um projeto, os itens da Fase 1 marcam zero sem nenhuma indicação de que o problema é o config, não o aluno. Por isso todo item do relatório exibe a "prova" (o caminho testado): se a lista der zero, o professor vê que testou "src/screens/Lista.js" quando o grupo chamava o arquivo de "TelaHome.js" e corrige o config, em vez de punir o grupo.

Parte 3 — Como aplicar no seu projeto

Esta é a parte executável. Se você leu até aqui, o resto é seguir a receita.

3.1 Pré-requisitos

- - Um repositório no GitHub — a esteira depende do GitHub Actions e não roda em GitLab nem em CI local.
 - Node.js na sua máquina, só para testar o script antes de abrir o PR (opcional, mas recomendado).
 - Permissão de escrita no repositório, ou um fork seu para abrir o PR.
- As fichas das fases da sua disciplina — elas são a fonte dos critérios. Sem elas você não tem checklist.

3.2 Passo a passo, do zero ao badge

Siga na ordem. Os passos 1 a 4 são feitos uma vez por projeto; os passos 5 a 8 se repetem em cada turma nova.

- 1 Crie um fork do projeto para enviar seus PRs**
Você não deve ter permissão de escrita no repositório do aluno. Envie sempre do seu fork pessoal, via pull request.
- 2 Escreva o config para aquele projeto**
Copie `pam-ci.config.json` para `.github/` e ajuste os caminhos (principal, lista, form, detalhe, dados, storage, scripts) e o campo `url`. Este é o passo que exige ler a estrutura do projeto do aluno — e é o único.
- 3 Copie os três scripts e o workflow**
`pam-ci.mjs`, `publicar-nota.mjs` e `pam-ci.yml` vão para `.github/scripts/` e `.github/workflows/` (veja a estrutura completa na seção 1.2). Nada precisa ser instalado — são arquivos `.mjs` e `.yml` puros.
- 4 Teste o script na sua máquina, contra o projeto real**
Dentro do clone do projeto do aluno, rode `node .github/scripts/pam-ci.mjs`. Ele imprime o relatório e grava `nota.md` e `nota.json`. Leia a nota: ela faz sentido? A Fase 1 está reconhecendo as telas que existem? Se um item der zero de forma suspeita, quase sempre é o config — confira a prova exibida e corrija o caminho.
- 5 Rode o publicador e confira o README**
Rode `node .github/scripts/publicar-nota.mjs`. Ele injeta o bloco do badge no README e cria o `NOTA.md`. Abra o README e confirme que só o bloco entre os marcadores mudou e que nada mais do documento do aluno foi tocado. Rode duas vezes: nada deve duplicar.
- 6 Limpe e faça o commit**
Apague `nota.md` e `nota.json` (são artefatos temporários locais; no repositório só entram `README.md` e `NOTA.md`), volte ao `.github/` e faça o commit com seu nome e e-mail de professor.
- 7 Crie um branch e envie**
Crie um branch (ex.: `ci/pam-nota`) e envie por SSH a partir do seu fork (veja a armadilha 1). Uma branch por projeto, sempre baseada no `main` atual do grupo.
- 8 Abra o pull request e explique ao grupo**
O corpo do PR deve dizer, em uma frase, que a ferramenta mede o escopo das fases e não mexe no código deles, e apontar onde a nota aparece. Alunos tendem a achar que é um mecanismo de vigilância; Sea transparente sobre a finalidade desde a primeira linha.

3.3 Adaptando às arquiteturas mais comuns

Cada grupo organiza os arquivos do seu jeito. O config absorve quase toda essa diferença. Os casos observados nos 12 projetos desta turma:

Arquitetura do aluno	O que ajustar no config	Observação
Expo clássico (App.js na raiz)	"principal": "App.js", "raiz": "."	O caso mais simples; a maioria dos grupos cai aqui
Expo Router (app/ na raiz)	"raiz": ".", telas em app/...	Aponte lista/form/detalhe para dentro de app/
Projeto dentro de subpasta	"raiz": "educamirim" (ou app/)	Todo caminho é relativo a essa raiz; 5 dos 12 grupos usavam
TypeScript (.tsx)	Nada nos critérios	O analisador já lê .ts e .tsx; nenhum ajuste necessário
Sem tela de detalhe ainda	Deixe "detalhe" apontando para onde deveria estar	O item da Fase 1 dá zero — correto, porque a fase pede

Regra de ouro da adaptação

Você configura a esteira para reconhecer a arquitetura que o aluno construiu. Você nunca muda a arquitetura do aluno para ela caber no validador. Se um critério não dá conta de uma estrutura legítima, ajuste o critério ou o config — nunca puxe o trabalho do grupo na sua direção.

3.4 Testes antes de abrir o PR

Um roteiro curto que pega quase tudo antes do grupo ver:

- ☐ Rodei o analisador e a nota faz sentido (nenhuma fase com zero inexplicável).
- ☐ Rodei o publicador e conferi que o README só mudou entre os marcadores.
- ☐ Rodei o publicador duas vezes: nada duplicou.
- ☐ Apaguei nota.md e nota.json antes do commit.
- ☐ O commit leva .github/, README.md e NOTA.md — e nada mais do aluno.
- ☐ A branch foi baseada no main atual do grupo (não em um fork atrasado).
- ☐ Enviei por SSH e confirmei que a branch existe no fork (git ls-remote).
- ☐ Abri o PR e o corpo dele está em português, explicando que a ferramenta não altera o código do grupo.

Parte 4 — Referência rápida

4.1 Os 40 critérios, item a item

A lista completa, como o analisador verifica. Guarde esta página: é o conteúdo que você vai querer ajustar quando a disciplina evoluir.

Fase 1 — Estrutura do projeto (10/10 pts) (1 ponto por item — total 10 pontos)

- ☐ +1 README.md existe e fala do projeto/grupo
- ☐ +1 Arquivo principal do app existe (App.js)
- ☐ +1 package.json existe com a dependência "expo"
- ☐ +1 Existe tela de LISTAGEM
- ☐ +1 Existem dados iniciais (seed) em arquivo de dados
- ☐ +1 Existe tela de FORMULÁRIO
- ☐ +1 Existe tela de DETALHE
- ☐ +1 Dependências importadas existem no package.json (app não quebra ao abrir)
- ☐ +1 app.json identifica o app (name/slug preenchidos)
- ☐ +1 Projeto tem pelo menos 2 arquivos de tela/código

Fase 2 — Persistência com AsyncStorage (15/15 pts) (1 ponto por item — total 15 pontos)

- ☐ +1 Dependência async-storage está no package.json
- ☐ +1 Existe import do AsyncStorage no código
- ☐ +1 Storage faz leitura com AsyncStorage.getItem
- ☐ +1 Storage grava com AsyncStorage.setItem
- ☐ +1 Existe função de CARREGAR a lista (carregar/load)
- ☐ +1 Existe função de ADICIONAR/CADASTRAR
- ☐ +1 Existe busca por id (buscar/find/getItem)
- ☐ +1 Existe função de EXCLUIR/remove
- ☐ +1 Formulário lê entradas com TextInput
- ☐ +1 Formulário salva chamando adicionar/salvar
- ☐ +1 Lista é alimentada a partir do storage
- ☐ +1 Tela de detalhe usa busca/dados do storage
- ☐ +1 Exclusão usa confirmação (Alert.alert)
- ☐ +1 Dados iniciais/seed são gravados na 1ª execução
- ☐ +1 Formulário valida campos (trim/length/vazio)

Fase 3 — Banco de dados SQLite (28/30 pts) (2 pontos por item — total 30 pontos)

- ☐ +2 Dependência expo-sqlite está no package.json
- ☐ +2 Existe import do expo-sqlite
- ☐ +2 Existe arquivo de banco de dados
- ☐ +2 Cria a tabela com CREATE TABLE IF NOT EXISTS
- ☐ +2 Insere dados com INSERT INTO

- ☒ +2 Consulta com SELECT
- ☒ +2 Atualiza com UPDATE
- ☒ +2 Exclui com DELETE FROM
- ☒ +2 Usa filtros com WHERE
- ☒ +2 Banco aberto com openDatabaseAsync/openDatabase
- ☒ +2 Tela de lista carrega dados do banco
- ☐ +2 Formulário salva no banco (INSERT/runAsync)
- ☒ +2 Detalhe busca no banco com WHERE/SELECT
- ☒ +2 Usa async/await corretamente (mais de 2 await)
- ☒ +2 Banco possui dados iniciais (seed inserido em SQL)

4.2 Comandos do dia a dia

Para	Comando
Rodar o analisador localmente	<code>node .github/scripts/pam-ci.mjs</code>
Publicar no README e no NOTA.md	<code>node .github/scripts/publicar-nota.mjs</code>
Ver a nota em JSON	<code>cat nota.json</code>
Rodar uma rodada manualmente no GitHub	Actions -> PAM I - Validacao das fases e nota -> Run workflow
Baixar o relatório da rodada	Aba Actions -> Artifacts -> nota-pam
Ver a nota no repositório	README.md (bloco no topo) ou NOTA.md
Conferir se a branch existe no fork	<code>git ls-remote git@github.com:voce/Repo.git</code>
Ver o badge funcionando	Abra o README do projeto em modo anônimo (sem estar logado)

4.3 Troubleshooting

Sintoma	Causa provável	O que fazer
Push rejeitado com "without workflow scope"	Token sem escopo de workflow	Envie por SSH (chave). Ver Armadilha 1
Script diz que enviou, mas a branch não existe	Erro do git escondido por um pipe	Rode git push sem pipe e confira; teste com git ls-remote
Workflow roda mas não escreve no README	Não é push na main, ou é PR	O publicador só roda em push na main. Em PR, o branch já sobe com o relatório
Infinite loop de commits	Faltou [skip ci] ou a checagem do autor	Inclua [skip ci] na mensagem e <code>github.actor != github-actions[bot]</code>
Fase inteira com zero	Config apontando para caminhos errados	Leia a "prova" de cada item: ela mostra o caminho testado. Corrija o config
Fase 1 dá zero mas o projeto tem telas	raiz errada no config	O config aponta para a subpasta errada; ajuste "raiz"
Nota parece errada num item	Critério por regex não reconheceu o nome do aluno	Ajuste a regex em pam-ci.mjs para aceitar o padrão da turma (é esperado)
Badge não muda depois do merge	O bot commita mas o workflow não roda	Veja se o commit do bot tem [skip ci] e se houve um push depois dele
Artefato com 0 downloads	Esperado — ninguém precisa baixar	O canal principal é o README. O artefato é só para quem quiser

Parte 5 — Limitações e cuidados

Uma ferramenta de nota precisa ser honesta sobre o que não sabe. Esta parte é o que você deve comunicar aos alunos e a si mesmo.

5.1 O que a nota mede

- Presença e coerência da estrutura do projeto (as três telas, o app principal, a identidade do app).
- Se as dependências estão declaradas — o único teste que pega erro de configuração real.
- Uso das APIs da Fase 2 (AsyncStorage) e da Fase 3 (SQLite) no código.
- A cadeia completa de CRUD na Fase 3, incluindo a integração com as telas.
- Presença de async/await, tratamento de erro por Alert, e validação de formulário.

5.2 O que ela não mede

- Se o app realmente roda. O analisador lê o código; ele não executa nada.
- Se o SQL está correto. Um SELECT sintaticamente válido e semanticamente errado ganha o mesmo ponto.
- Qualidade visual, layout, acessibilidade, design.
- Trabalho em equipe. Quem fez o quê não é visível no código de forma confiável.
- Sobrevivência (copiar do colega, do StackOverflow, de um projeto anterior). A esteira não é antifraude; um SQL copiado da internet passa.

5.3 Falsos positivos e falsos negativos

Sendo uma análise por regex, os dois erros são possíveis:

Erro	Exemplo	Com que frequência ocorre
Falso negativo (aluno injustamente penalized)	O aluno deu o nome buscarProduto à função em vez de findItem; o padrão não casou	Comum no início; ajuste a regex para a convenção da sua turma
Falso positivo (aluno ganha ponto de graça)	Comentário com // TODO: SELECT FROM aqui marca o item de SELECT	Menos comum, mas acontece; por isso a prova é exibida

Como usar a nota sem injustiças

A nota é um ponto de partida para a conversa, não um veredito. Todo item traz a prova (o arquivo e o padrão testados): quando um zero parece errado, quase sempre é o config ou a regex — não o aluno. Use a nota para ordenar as conversas, não para encerrá-las.

5.4 Checklist de revisão humana

Antes de fechar a avaliação de um projeto:

- ☐ Rodei a esteira e a nota saiu coerente com o que eu sei do projeto.
- ☐ Para cada zero que me pareceu estranho, li a prova e ajustei o config (não o aluno).
- ☐ Para cada nota alta com projeto fraco, olhei o código — a regex pode ter sido enganada.

- ☐ Conferi que nenhum commit alterou código do aluno (só README e NOTA.md).
- ☐ Consideri a participação e o processo, que a nota não mede.

Apêndice A — Os 12 projetos e suas notas

Resultado da primeira rodada completa da esteira, com as notas medidas no repositório de cada grupo. Serve de referência para calibrar a sua turma: compare com a sua e veja se a distribuição faz sentido.

Distribuição

Conceito	Quantidade	Projetos
MB — Muito bom	2	EDUCAMIRIM, CDF
B — Bom	2	Boom-Patch, equipcontrol
R — Regular	8	EduSophia, Vamo-a-, F.O.C, ToDoDev-, 3GDStock, TripGo, Pet_Match, MEIHub
I — Insuficiente	0	(nenhum)

Nota por projeto

#	Projeto	Grupo	F1	F2	F3	Nota	%	Pontos	PR
1	EDUCAMIRIM	VIISRE	10/10	15/15	28/30	MB	96%	53/55	#5 merged
2	CDF	Controle Fin. Diário	9/10	14/15	26/30	MB	89%	49/55	#5 aberto
3	Boom-Patch	TRP bentão	9/10	15/15	6/30	B	55%	30/55	#9 merged
4	equipcontrol	TechControl	10/10	13/15	8/30	B	56%	31/55	#5 aberto
5	EduSophia	Educa+	9/10	13/15	4/30	R	47%	26/55	#4 aberto
6	Vamo-a-	ref's	9/10	15/15	2/30	R	47%	26/55	#7 merged
7	F.O.C	F.O.C	8/10	13/15	4/30	R	45%	25/55	#6 merged
8	ToDoDev-	FocusIT	8/10	12/15	2/30	R	40%	22/55	#5 aberto
9	3GDStock	3GD	8/10	10/15	4/30	R	40%	22/55	#5 aberto
10	TripGo	TripGo	8/10	10/15	2/30	R	36%	20/55	#8 merged
11	Pet_Match	Zenin's	7/10	11/15	2/30	R	36%	20/55	#8 merged
12	MEIHub	Babuínos do note.js	8/10	10/15	2/30	R	36%	20/55	#6 aberto

O que a distribuição revela

O dado mais informativo desta tabela não é a média — é a coluna da Fase 3. Ela mostra que 7 dos 12 grupos entregaram as fases 1 e 2 de forma razoável, mas pararam antes de fechar a Fase 3 (SQLite):

- A Fase 3 é o maior gap da turma. Onde o grupo investe em SQLite, a nota sobe muito — CDF e EDUCAMIRIM mostram que só pela Fase 3 dá para chegar ao MB.
- Nenhum projeto ficou no I. A base (Fases 1 e 2) está bem consolidada na turma; o que falta é o conteúdo novo.

- Dois grupos (equipcontrol e Boom-Patch) estão no B já na primeira rodada, e um deles tinha o CI rodando há semanas — o que mostra que a nota responde ao esforço, como deve ser. Estes dados foram coletados com a esteira funcionando, mas antes de os grupos terem os PRs de nota mergeados. Os números de cada projeto atualizam sozinhos no README de cada repositório a cada push — se você copiar esta tabela, ela envelhece. Consulte os repositórios para o valor atual.